

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 21.Б07-мм

Валидация спецификации RISC-V ISA в LLVM относительно Sail

Хечнев Семен Евгеньевич

Отчёт по преддипломной практике
в форме «Решение»

Научный руководитель:
инженер-исследователь лаборатории технологий программирования инфраструктурных
решений, Косарев Д. С.

Санкт-Петербург
2025

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. RISC-V	5
2.2. Sail	5
2.3. LLVM TableGen	7
2.4. Прототип анализатора	8
3. Особенности реализации	9
3.1. Сопоставление узлов AST и мнемоник	9
3.2. Поиск входных и выходных операндов	9
3.3. Анализ обращения инструкции к памяти	10
3.4. Особенности расширения «P» в sail-riscv-p	11
3.5. CI	12
4. Найденные расхождения	13
Заключение	15
Список литературы	16

Введение

Архитектура набора команд (ISA) RISC-V [4] на сегодняшний день является одной из наиболее активно развивающихся открытых архитектур. Её открытость, модульность и поддержка со стороны научного и индустриального сообществ способствуют её распространению в широком спектре систем — от встраиваемых платформ до высокопроизводительных процессоров.

Важную роль в экосистеме RISC-V играет проект LLVM [3], в котором спецификация RISC-V ISA описана с помощью декларативного языка TableGen [2]. Параллельно, существует одобренный RISC-V Foundation эмулятор `sail-riscv`¹, написанный на языке Sail [5]. Данный эмулятор используется для верификации, моделирования и формального анализа, обеспечивая строгую интерпретацию поведения инструкций на уровне архитектуры. Таким образом, одновременно существуют две ключевые спецификации RISC-V ISA, каждая из которых описывает одни и те же инструкции, но с разных позиций.

Несмотря на наличие двух независимых спецификаций RISC-V ISA, в настоящее время не существует инструментов, позволяющих систематически сопоставлять эти спецификации и проверять их согласованность. Это создаёт риск накопления расхождений между декларативным описанием в LLVM и формальной моделью на Sail по мере их эволюции. Такие расхождения могут затрагивать синтаксис инструкций, ограничения на операнды, семантические свойства или поведение в особых ситуациях. Наличие подобных расхождений может приводить к некорректной генерации кода компилятором, расхождению поведения аппаратных реализаций и снижению общей надёжности экосистемы RISC-V.

Настоящая работа посвящена валидации декларативной спецификации RISC-V ISA в LLVM относительно её формальной модели на Sail. В работе предлагается разработать анализатор, который будет автоматически искать несогласованности спецификаций на Sail и LLVM.

¹<https://github.com/riscv/sail-riscv> (дата обращения: 17 мая 2025 г.).

1. Постановка задачи

Целью работы является валидация спецификации RISC-V ISA в LLVM относительно Sail. Для её выполнения были поставлены следующие задачи:

1. разработать прототип анализатора, который будет включать следующие анализы:
 - поиск входных и выходных операндов инструкций;
 - определение, обращается ли инструкция к памяти;
2. при помощи анализатора сравнить следующие спецификации RISC-V ISA:
 - LLVM v19 против sail-riscv²;
 - «P» extension (Version 0.9.11-draft-20211209³) в LLVM против sail-riscv-p⁴;
 - для sail-riscv-p дополнительно определить для каждой инструкции какие CSR регистры читает и пишет;
3. протестировать анализатор.

²<https://github.com/riscv/sail-riscv/commits/05b845c91d1c1db7b361fc8d06e815b54ca0b07a> (дата обращения: 17 мая 2025 г.).

³<https://tools.cloudbear.ru/docs/riscv-p-extension-0.9.11-20211209.pdf> (дата обращения: 17 мая 2025 г.).

⁴<https://github.com/umcann123/sail-riscv-p> (дата обращения: 17 мая 2025 г.).

2. Обзор

2.1. RISC-V

RISC-V — свободная и открытая архитектура процессоров, впервые представленная в 2010 году в университете Беркли, США [6]. Создана по концепции RISC (Reduced Instruction Set Computer) [1], то есть обладает сравнительно небольшим набором инструкций с возможностью дополнения с помощью расширений.

2.2. Sail

Эмулятор архитектуры — это инструмент, который позволяет исполнять программу для одной архитектуры на процессоре другой (или той же) архитектуры. Sail — язык для определения эмуляторов процессорных архитектур. На языке Sail реализованы модели множества архитектур, включая X86, Arm, RISC-V⁵ и многие другие. По моделям на данном языке Sail умеет генерировать эмулятор на OCAML или C. Кроме генерации эмуляторов, Sail также позволяет верифицировать модели ISA, генерировать и исполнять автоматические тесты, а также исполнять тесты безопасности кода. Сам компилятор эмулятора Sail написан на OCAML.

2.2.1. Модель sail-riscv

Sail-riscv — это формальная модель RISC-V ISA, написанная на Sail. Сама модель представляет из себя интерпретатор. Во-первых, для всех инструкций задаётся Abstract Syntax Tree. Причём один узел AST может представлять множество инструкций. Например, на листинге 1 узел ITYPE, который параметризован 12-ти битным непосредственным (immediate) значением, индексами выходного и входного регистров и интерпретируемой операцией, представляет инструкции `addi`, `slti`, `sltiu`, `xori`, `ori` и `andi`.

⁵<https://github.com/riscv/sail-riscv> (дата обращения: 17 февраля 2025 г.).

Листинг 1: Пример определения узла AST в sail-riscv

```
enum iop = {RISCV_ADDI, RISCV_SLTI, RISCV_SLTIU,  
            RISCV_XORI, RISCV_ORI, RISCV_ANDI}  
  
union clause ast = ITYPE : (bits(12), regidx, regidx, iop)
```

Затем для каждого узла AST определяются функции `encdec` — кодирования и декодирования 32-ух битного кода операции (`opcode`) и AST.

Листинг 2: Пример определения функции кодирования и декодирования инструкций в sail-riscv

```
mapping encdec_iop : iop <=> bits(3) = {  
  RISCV_ADDI <=> 0b000,  
  RISCV_SLTI <=> 0b010,  
  RISCV_SLTIU <=> 0b011,  
  RISCV_ANDI <=> 0b111,  
  RISCV_ORI <=> 0b110,  
  RISCV_XORI <=> 0b100  
}  
  
mapping clause encdec = ITYPE(imm, rs1, rd, op)  
  <=> imm @ rs1 @ encdec_iop(op) @ rd @ 0b0010011
```

Код, который отвечает за интерпретацию инструкции содержится в `execute` функциях, которые в качестве аргумента содержат интерпретируемый узел AST.

Листинг 3: Пример определения семантики исполнения инструкций `addi`, `slti`, `sltiu`, `andi`, `ori` и `xori` на Sail

```
function clause execute (ITYPE (imm, rs1, rd, op)) = {  
  let rs1_val = X(rs1); // read the rs1  
  let immext : xlenbits = EXTS(imm); // sign-extend imm  
  let result : xlenbits = match op { // compute the result  
    RISCV_ADDI => rs1_val + immext, // for ADDI, do an addition
```

```

RISCV_SLTI  => EXTZ(rs1_val <_s immext), // ...etc
RISCV_SLTIU => EXTZ(rs1_val <_u immext),
RISCV_ANDI  => rs1_val & immext,
RISCV_ORI   => rs1_val |  immext,
RISCV_XORI  => rs1_val ^  immext
};
X(rd) = result;                // write result to rd
RETIRE_SUCCESS                  // successful termination
}

```

Наконец, для каждого узла AST определяются функции `assembly`, которые отображают узлы AST в строки (мнемоники).

Листинг 4: Пример определения функции `assembly` для узла `ITYPE` на Sail

```

mapping itype_mnemonic : iop <=> string = {
  RISCV_ADDI  <=> "addi",
  RISCV_SLTI  <=> "slti",
  RISCV_SLTIU <=> "sltiu",
  RISCV_XORI  <=> "xori",
  RISCV_ORI   <=> "ori",
  RISCV_ANDI  <=> "andi"
}

mapping clause assembly = ITYPE(imm, rs1, rd, op)
  <=> itype_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep()
    ^ reg_name(rs1) ^ sep() ^ hex_bits_signed_12(imm)

```

2.3. LLVM TableGen

LLVM — это инфраструктура для разработки компиляторов, созданная в 2003 году в Иллинойском университете. LLVM поддерживает широкий спектр целевых платформ, включая популярные x86 и ARM, так и молодые архитектуры, такие как RISC-V. Важную роль в бэкендах

LLVM играет TableGen.

TableGen — это декларативный язык, используемый в LLVM для многих целей. В частности TableGen используется для описания ISA целевой архитектуры. Информация из TableGen файлов используется в бэкендах LLVM. Файлы TableGen имеют расширение .td. Инструментами LLVM можно .td файлы скомпилировать в JSON.

Листинг 5: Описание инструкции addi при помощи TableGen

```
let hasSideEffects = 0, mayLoad = 0, mayStore = 0 in
class ALU_ri<bits<3> funct3, string opcodestr>
  : RVInstI<funct3, OPC_OP_IMM, (outs GPR:$rd),
    (ins GPR:$rs1, simm12:$simm12),
    opcodestr, "$rd, $rs1, $simm12">,
    Sched<[WriteIALU, ReadIALU]>;

let isReMaterializable = 1, isAsCheapAsAMove = 1 in
def ADDI : ALU_ri<0b000, "addi">;
```

В описании инструкции на TableGen может, например, постулироваться, что некоторые команды процессора имеют соответствующие входные и выходные регистры, (не)могут писать в память, являются (не)коммутативными, сложность операций и т.п.

2.4. Прототип анализатора

Для исследования предметной области моим научным руководителем уже была начата разработка анализатора для валидации. Данные в анализатор из LLVM поступают в виде JSON, код на Sail анализируется как есть. Сам Sail реализован на языке OCAML, и анализатор использует компилятор Sail как библиотеку. Анализатор умел находить исполняемые спецификации для инструкций RISC-V и запускать на них тривиальные анализы, а именно наивную проверку выходных аргументов инструкций.

3. Особенности реализации

Как было сказано ранее эмулятор `sail-riscv` представляет из себя интерпретатор некоторого AST, каждый узел которого параметризован некоторыми параметрами. Причём всякая специализация узла AST представляет из себя конкретную инструкцию. Например, специализация узла листинга 1 ITYPE (`imm`, `rs1`, `rs2`, `RISCV_ADDI`) представляет из себя инструкцию `addi rd, rs1, imm`. Специализировать имеет смысл только параметры типа `bool` или перечисления (`enum`).

3.1. Сопоставление узлов AST и мнемоник

Во-первых, необходимо было определить за какие инструкции отвечает каждая специализация узла AST в эмуляторе. Для этого производился анализ специализаций функций `assembly`, которые как раз отвечают за отображение узлов AST в мнемоники инструкций.

3.2. Поиск входных и выходных операндов

В модели `sail-riscv` для взаимодействия с регистрами используются следующие функции:

- `rX` для чтения, `wX` для записи GPR регистров;
- `rF` для чтения, `wF` для записи вещественных регистров;
- `rV` для чтения, `wV` для записи векторных регистров.

Для поиска входных и выходных операндов строился граф вызовов всех функций для всех специализаций функций `execute`, которые отвечают за интерпретацию поведения инструкций. Затем производился обход этого графа вызовов и определение того, какие функции с какими аргументами вызывали функции для взаимодействия с регистрами.

Дополнительно для каждой функции производился анализ алиасов⁶.

⁶https://en.wikipedia.org/wiki/Alias_analysis (дата обращения: 17 декабря 2024 г.).

Листинг 6: Пример функции, которая интерпретирует выполнение инструкции `c.addi4spn`

```
function clause execute (C_ADDI4SPN(rdc, nzimm)) = {  
    let imm : bits(12) = (0b00 @ nzimm @ 0b00);  
    let rd = creg2reg_idx(rdc);  
    execute(ITYPE(imm, sp, rd, RISCV_ADDI))  
}
```

Если не производить анализ алиасов, то, например, на листинге 6 входным регистром был бы регистр `rd`, но на самом деле входным регистром является регистр `rdc`.

3.3. Анализ обращения инструкции к памяти

Для работы с памятью в `sail-riscv` используются функции `read_ram` и `write_ram`. Для того чтобы определить читает или пишет инструкция память, для каждой специализации функции `execute` проверялась достижимость вершин `read_ram` и `write_ram` в ранее определённом графе вызовов всех функций.

Однако любая инструкция, которая работает с адресами, для трансляции виртуального адреса в физический обращалась к таблице виртуальных адресов. И при чтении таблицы виртуальных адресов могла происходить как запись в память, так и чтение из памяти.

- Чтение из памяти могло происходить в случае, если при трансляции виртуального адреса в физический происходил TLB miss. Тогда необходимо было прочитать таблицу виртуальных адресов из памяти.
- Запись в память могла происходить тогда, когда при трансляции виртуального адреса в физический необходимо было обновить бит A или D у записи таблицы виртуальных адресов.

По требованию заказчика обращения к памяти при трансляции адресов игнорировались. Для этого при решении задачи достижимости

вершин `read_ream` и `write_ram` в графе вызовов игнорировался вызов функции `translateAddr`, отвечающей за трансляцию виртуального адреса в физический.

3.4. Особенности расширения «P» в `sail-riscv-p`

Проект `sail-riscv-p`⁷ — это форк эмулятора `sail-riscv` с реализованным расширением «P» (packed SIMD).

Листинг 7: Пример функции `execute`, которая может интерпретировать регистры как парные

```
function clause execute (PEXT_64BITS_ADD_SUB(rs2, rs1, rd, op))
= {
  results : bits(64) = zeros();
  rs1_val : bits(64) = zeros();
  rs2_val : bits(64) = zeros();
  if sizeof(xlen) == 32
  then {
    rs1_val = X(rs1 | 0b00001) @ X(rs1 & 0b11110);
    rs2_val = X(rs2 | 0b00001) @ X(rs2 & 0b11110);
  }
  else {
    rs1_val = X(rs1);
    rs2_val = X(rs2);
  };
  results = match op {
    ADD64    => rs1_val + rs2_val,
    ... \\ etc
  };
  if sizeof(xlen) == 32
  then {
    X(rd | 0b00001) = results[63..32];
    X(rd & 0b11110) = results[31..0];
  }
```

⁷<https://github.com/umcann123/sail-riscv-p> (дата обращения: 17 декабря 2024 г.).

```

    }
    else X(rd) = results;
    RETIRE_SUCCESS
}

```

На листинге 7 видно, что функция `PEXT_64BITS_ADD_SUB` зависит от `xlen` — разрядности RISC-V, и в зависимости от `xlen` функция `execute` работает с регистрами по разному. В случае если `xlen` равен 32, то входные и выходные регистры интерпретируются как парные, иначе, если `xlen` не равно 32, то входные и выходные регистры интерпретируются как обычно.

Чтобы определить интерпретирует ли инструкция регистры как парные, производилось приведение программы к ANF⁸ форме, а затем строился и анализировался data flow graph.

Во-первых, для того чтобы аргументами всех функций были только тривиальные значения (константы и переменные) было реализовано приведение представления программы к ANF форме.

Листинг 8: Вызов чтения из регистра до ANF

```
X(rs1 | 0b000001)
```

Листинг 9: Вызов чтения из регистра после ANF

```
let a1 = rs1 | 0b000001 in X(a1)
```

Затем анализировался data flow graph каждой нормализованной `execute` функции.

3.5. CI

Был добавлен CI, который проверяет успешность сборки анализатора и проход тестов.

⁸https://en.wikipedia.org/wiki/A-normal_form (дата обращения: 17 декабря 2024 г.)

4. Найденные расхождения

Для LLVM версии 19 и sail-riscv⁹ при помощи анализатора были найдены следующие расхождения:

1. `sp` не является явным аргументом в sail-riscv для инструкций: `c.addi16sp`, `c.addi4spn`, `c.fldsp`, `c.flwsp`, `c.fsdsp`, `c.fswsp`, `c.ldsp`, `c.lwsp`, `c.sdsp`, `c.swsp`.
2. `RA` является неявным out регистром в sail-riscv для инструкций: `c.jal`, `c.jalr`
3. В sail-riscv `vd` — входной регистр для инструкций: `vadc.vim`, `vadc.vvm`, `vadc.vxm`, `vcompress.vm`, `vfmerge.vfm`, `vmadc.vi`, `vmadc.vim`, `vmadc.vv`, `vmadc.vvm`, `vmadc.vx`, `vmadc.vxm`, `vmand.mm`, `vmandn.mm`, `vmerge.vim`, `vmerge.vvm`, `vmerge.vxm`, `vmnand.mm`, `vmnor.mm`, `vmor.mm`, `vmorn.mm`, `vmsbc.vv`, `vmsbc.vvm`, `vmsbc.vx`, `vmsbc.vxm`, `vmxnor.mm`, `vmxor.mm`, `vsbc.vvm`, `vsbc.vxm`.
4. В sail-riscv `vd` и (неявно) `v0` — входные регистры для инструкций: `vaaddu.vv`, `vaaddu.vx`, `vaadd.vv`, `vaadd.vx`, `vadd.vi`, `vadd.vv`, `vadd.vx`, `vand.vi`, `vand.vv`, `vand.vx`, `vasubu.vv`, `vasubu.vx`, `vasub.vv`, `vasub.vx`, `vdivu.vv`, `vdivu.vx`, `vdiv.vv`, `vdiv.vx`, `vfadd.vf`, `vfadd.vv`, `vfclass.v`, `vfcvt.f.xu.v`, `vfcvt.f.x.v`, `vfcvt.rtz.x.f.v`, `vfcvt.rtz.xu.f.v`, `vfcvt.x.f.v`, `vfcvt.xu.f.v`, `vfddiv.vf`, `vfddiv.vv`, `vffirst.m`, `vfmacc.vf`, `vfmacc.vv`, `vfmadd.vf`, `vfmadd.vv`, `vfmax.vf`, `vfmax.vv`, `vfmin.vf`, `vfmin.vv`, `vfmsac.vf`, `vfmsac.vv`, `vfmsub.vf`, `vfmsub.vv`, `vfmul.vf`, `vfmul.vv`, `vfmv.s.f`, `vfmv.v.f`, `vfncvt.f.f.w`, `vfncvt.f.xu.w`, `vfncvt.f.x.w`, `vfncvt.rod.f.f.w`, `vfncvt.rtz.x.f.w...`¹⁰.

Для реализации расширения «P» в LLVM и в sail-riscv-p¹¹ при помощи анализатора были найдены следующие расхождения:

⁹<https://github.com/riscv/sail-riscv/commits/05b845c91d1c1db7b361fc8d06e815b54ca0b07a> (дата обращения: 17 мая 2025 г.).

¹⁰Полный список разхождений — https://github.com/s-khechnev/llvm_td_lint_playground/blob/ins_outs_regs/Results.md (дата обращения: 12 марта 2025 г.).

¹¹<https://github.com/umcann123/sail-riscv-p> (дата обращения: 17 декабря 2024 г.).

- В sail-riscv-p rd является входным регистром для инструкций:
ave, bitrev, kaddw, kdmbb, kdmbb16, kdmbt, kdmbt16, kdmtt,
kdmtt16, khmbb16, khmbt16, khmtt16, kmdda, kmmwb2, kmmwb2.u,
kmmwt2, kmmwt2.u, kmxda, ksllw, kslraw, kslraw.u, ksubw, kwmmul,
kwmmul.u, smbb16, smbt16, smdrs, smds, smmul, smmul.u, smmwb,
smmwb.u, smmwt, smmwt.u, smtt16, smxds, sra.u, ukaddw, uksbw

Заключение

Таким образом, в рамках преддипломной практики были достигнуты следующие результаты:

- разработан прототип анализатора, который реализует следующие анализы;
 - поиск входных и выходных операндов инструкций;
 - определение, обращается ли инструкция к памяти;
- при помощи анализатора сравнены следующие спецификации RISC-V ISA:
 - LLVM v19 против sail-riscv¹²;
 - «P» extension (Version 0.9.11-draft-20211209¹³) в LLVM против sail-riscv-p¹⁴;
 - * для sail-riscv-p дополнительно определены для каждой инструкции какие CSR регистры читает и пишет;

Исходный код анализатора: https://github.com/s-khechnev/llvm_td_lint_playground.

Имя коммитера: s-khechnev.

¹²<https://github.com/riscv/sail-riscv/commits/05b845c91d1c1db7b361fc8d06e815b54ca0b07a> (дата обращения: 17 мая 2025 г.).

¹³<https://tools.cloudbear.ru/docs/riscv-p-extension-0.9.11-20211209.pdf> (дата обращения: 17 мая 2025 г.).

¹⁴<https://github.com/umcann123/sail-riscv-p> (дата обращения: 17 мая 2025 г.).

Список литературы

- [1] Jamil Tariq. Risc versus cisc // Ieee Potentials. — 1995. — Vol. 14, no. 3. — P. 13–16.
- [2] LLVM TableGen. — URL: <https://llvm.org/docs/TableGen/> (дата обращения: 16 декабря 2024 г.).
- [3] Lattner Chris, Adve Vikram. LLVM: A compilation framework for life-long program analysis & transformation // International symposium on code generation and optimization, 2004. CGO 2004. / IEEE. — 2004. — P. 75–86.
- [4] The RISC-V instruction set manual, volume I: User-level ISA, version 2.0 / Andrew Waterman, Yunsup Lee, David A Patterson, Krste Asanovic // EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-54. — 2014. — P. 4.
- [5] The Sail instruction-set semantics specification language : Rep. / Technical report published by Cambridge University ; Executor: Kathryn E Gray, Peter Sewell, Christopher Pulte et al. : 2017.
- [6] Waterman Andrew Shell. Design of the RISC-V instruction set architecture. — University of California, Berkeley, 2016.